

Управление разработкой корпоративных информационных систем

Основные понятия и определения. Жизненный цикл

В этом модуле мы продолжим рассказ о Моделях жизненного цикла корпоративных систем. А именно, мы рассмотрим Основы жизненного цикла

О чем пойдет речь:

- Понятие ЖЦ
- Этапы ЖЦ
- Обзор моделей ЖЦ

Вначале речь пойдет о Понятии ЖЦ. Затем мы обсудим Этапы ЖЦ. Потом перейдем к Обзору моделей ЖЦ.

И, наконец, подведем итоги и сделаем предварительные выводы".

ЖЦ разработки ПО

«Любая разработка ПО происходит по «жизненному циклу», состоящему из всех видов деятельности, которые осуществляются с того момента, как версия 1.0 системы начинает свое существование лишь как идея, и до того момента, когда версия 6.74b сделает свой последний вздох на компьютере последнего заказчика.»

*Стив Мак Коннелл, Быстрая разработка, 1996
(Steve McConnell, RapidDevelopment)*

Начнем с того, что в общем смысле, жизненный цикл (ЖЦ) – это изменение состояния объекта т.е. его свойств и характеристик во временном диапазоне: от начала его существования и до момента исчезновения.

ЖЦ есть у всех объектов, которые не вечны. И кроме того, ЖЦ есть у всех процессов. Так, например, физики не одно столетие бьются над тем, как узнать, с чего началось зарождение Вселенной, и как именно она может прекратить существование. Они считают, что если есть процесс, а вселенная изменяется с течением времени, увеличивается в размерах, движется, то где-то есть начало этого процесса и где-то есть конец этого процесса.

Точно также у программ есть свой ЖЦ – от момента создания, до вывода из эксплуатации.

В контексте того, что мы будем говорить о стандартах ISO, которые вводят свое определение ЖЦ (Life Cycle), нужно быть предельно точным в употреблении терминов.

ЖЦ разработки ПО – особенности

- Программная система разрабатывается постепенно и развивается, начиная от зарождения идеи ПО до реализации и сдачи пользователю, и далее.
- Каждый этап завершается разработкой части системы или связанной с ней документации (план тестирования, руководство пользователя и т.д).
- Теоретически, для каждого этапа четко определены начальные и конечные точки, а также известно, что он должен передать следующему этапу.
- На практике все сложнее.

Программная система разрабатывается постепенно и развивается, начиная от концепции, от абстрактной идеи, и постепенно конкретизируется до программного продукта, который включает в себя не только код, но и большое количество документации – это и диаграммы, и документация к коду, и документация для пользователей по работе с программным продуктам, документация для администраторов для настройки, установки и сопровождения . Программные системы заканчиваются на этапе вывода из эксплуатации, который завершает сопровождение. Каждый этап ЖЦ завершается разработкой некоторой части системы – она может быть полнофункциональной, либо не совсем полнофункциональной. Это зависит от конкретной модели ЖЦ. Каждый этап завершается созданием документации. Документация включает более глобальные артефакты, такие как план проекта, план тестирования, план реализации, план сопровождения, или более узкие документы, такие как сценарии использования, руководство пользователя, руководство администратора, краткое руководства пользователя, основные требования к проекту, или более детальные требования в форме технического задания.

Границы фаз ЖЦ могут изменяться (также и динамически) в зависимости от модели ЖЦ и подхода к реализации

Цели изучения ЖЦ

- организация и управление разработкой ПО.
- основа для анализа разработки ПО.
- основа для планирования разработки ПО.
- корректная постановка процессов разработки ПО
- Анализ ЖЦ обязателен для сложных проектов.

Изучение жизненного цикла корпоративных программных систем необходимо, прежде всего, для понимания организации разработки ПО, то есть всех процессов, которые связаны с ЖЦ. Не поняв того, как устроен ЖЦ вообще, нельзя говорить о сколько-нибудь планомерной организации и

управлении этим процессом. Конечно, из успешных проектов нужно делать выводы и тиражировать те принципы, которые привели к успеху, те практические шаги и методы, которые дают возможность эффективно и планомерно развивать проекты, совершенствовать программные продукты, улучшать взаимодействие с пользователями, производство документации и все те процессы, которые лежат в основе жизненного цикла. Это следует делать на основе анализа результатов работы над предыдущими проектами и тут может помочь и план проекта, и другие глобальные документы – план тестирования, интеграции, реализации, сопровождения, а также вся проектная документация, которая была созданы. Кроме того - журналы ошибок, и те документы, которые создаются на этапе сопровождения программного продукта. В этом смысле, изучение ЖЦ дает важную основу для анализа разработки ПО, которая позволяет более тщательно планировать и производить те процессы, которые лежат в основе ЖЦ и тиражировать таким образом внутреннюю методологию, которая будет разработана и развита командой разработчиков. Таким образом можно прийти к корректной и адекватной постановке и адаптации. Безусловно, для каждого программного проекта, как правило, приходится иметь дело с некой стандартной методикой, методологией, вобравшей весь предыдущий опыт, который необходимо адаптировать с учетом характера и масштаба проекта к конкретному заказчику и конкретным условиям производства продукта. Важно помнить, что у заказчика имеется определенная и, как правило, уникальная комбинация программно-аппаратной среды, в которую предстоит интегрировать программный продукт. Это особенно важно с учетом корпоративных систем, поскольку это обуславливает большое количество взаимосвязей, объем, сложность программного окружения и в целом корпоративного программного комплекса. Таким образом, анализ ЖЦ является необходимой стадией для крупных и сложных программных проектов, каковым является КИС.

Жизненный цикл программной системы – это непрерывный, итерационный, замкнутый процесс

Также хочется отметить, что анализ ЖЦ обязателен для сложных проектов. Он нужен всегда, где мы имеем дело со сложным объектом, который сразу осознать нельзя: мы разделяем объект на более мелкие части, которые проще понимать, и которыми легче оперировать.

Знание жизненного цикла помогает в решении задач анализа, планирования и управления разработкой ПО

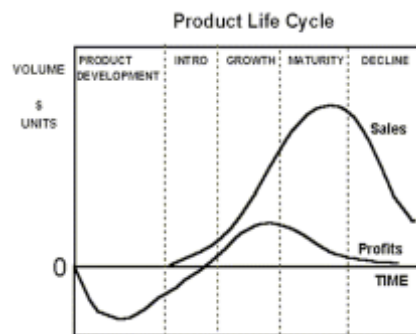
Связь ЖЦ продукта и проекта



Следует различать жизненный цикл проекта и жизненный цикл продукта. Жизненный цикл продукта может быть намного больше жизненного цикла проекта. Основные стадии жизненного цикла продукта -разработка, производство, эксплуатация- могут включать в себя много самостоятельных проектов:например, собственно разработка продукта, разработка его маркетинговой стратегии.

Экономика ЖЦ программного продукта

- Создание
- Вывод на рынок
- Рост
- Зрелость
- Упадок



Если говорить подробнее об экономике ЖЦ программного продукта, то нужно сказать, что он проходит целый ряд стадий и эти стадии вносят различный вклад в прибыль и динамику продаж. И, если на стадии создания и вывода на рынок проект, преимущественно, находится в минусе по прибылям, то после вывода на рынок, когда наблюдается рост и зрелость, прибыль становится положительной. В период зрелости, как правило, прибыль имеет отрицательный прирост, но положительное значение. В районе упадка уже наблюдается существенное падение и невысокое значение как прибыли, так и продаж, с точки зрения их динамики. Все это характерно, как для коробочного продукта, когда речь идет о количестве инсталляций, так и в случае, когда взаимодействие осуществляется с конкретным заказчиком, с учетом всех классов пользователей для продукта.

Рассмотрим более подробно экономику ЖЦ на основе сопоставления критериев развития, скорости роста бизнеса и доли рынка, которую занимает программный продукт. Здесь, в начале пути, нужны инвестиции, поскольку не известна дальнейшая судьбе программного продукта. Затем, программный продукт выходит на рынок, приносит доход и, наконец, приносит существенную прибыль. При этом без значительных затрат на поддержку продаж. Через некоторое время наступает этап, когда доходы относительно невысоки и продажи влекут за собой существенные затраты.

Вклад фаз ЖЦ в сроки и стоимость проекта:

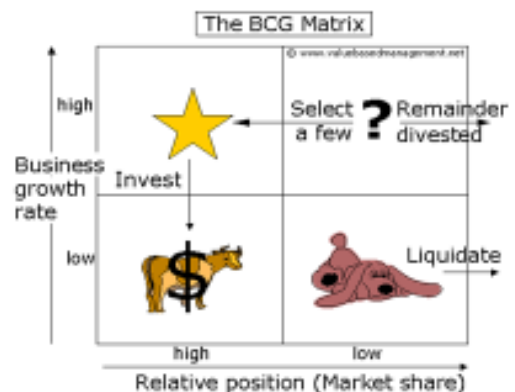
Фаза	Стоимость	Сроки
Требования (со спецификациями)	2%	21% / 18%
Спецификации	5%	см. «Требования»
Проектирование	6%	18% / 19%
Кодирование (с тестированием)	5%	36% / 34%
Тестирование	7%	см. «Кодирование»
Интеграция	8%	24% / 29%
Сопровождение	67%	---

Достаточно интересным является взгляд на вклад различных фаз ЖЦ программных проектов в сроки и стоимость. Анализ произведен на основе целого ряда проектов (около 1000), которые велись компанией НР и др. Очевидно, что сопровождение составляет львиную долю стоимости и сроков проекта. При этом такие стадии, как кодирование, даже вместе с тестированием, и анализ требований, даже вместе с изготовлением спецификаций, занимают относительно небольшую долю стоимости. Можно сделать ряд интересных выводов на основе анализа этой динамики. Основные затраты выделяются на сопровождение. Особенно это важно для корпоративных долгосрочных проектов, включающих большое количество компонентов, которые нужно объединять, интегрировать и поддерживать совместно, что вызывает дополнительные сложности. Кроме того, программные средства, увеличивающие расширяемость применения ПО, более эффективны, чем все попытки рефакторинга улучшения кода. Еще один интересный вывод состоит в том, что фазы перед кодированием и после него составляют порядка 30% затрат, а собственно кодирование при этом составляет всего 5%. То есть то, что называется программированием, для больших проектов отнюдь не является затратной частью. Обрамляющие стадии (тестирование и коррекция спецификаций) обеспечивают существенное улучшение качества ПО и его соответствие требованиям. Отметим, что правильная постановка обрамляющих стадий очень важна и

ускоряет кодирование. Большая часть серьезных ошибок, которые выявляются в программных проектах, происходит на стадиях проектирования и построения спецификаций. Поэтому эти ошибки очень дорогостоящи, поскольку приходится переделывать и код, и документацию, а также нужны формальные методы их анализа (это и ревизия проекта, и более формальные логические технологии проверки корректности). Цена ошибки растет примерно экспоненциально по мере продвижения проекта по жизненному циклу. Если ошибка обнаруживается на стадии анализа требований, то ее исправление достаточно дешево. Если же она обнаружена на более поздних стадиях, особенно на стадии сопровождения, то ее цена во много раз выше, потому что приходится изменять всю версию ПО, документацию, диаграммы и целый ряд других программных продуктов. К счастью, есть специальные средства для поиска, выявления и устранения ошибок.

Экономика ЖЦ ПО: «Бостонская матрица»

- «Знак вопроса»
 - Не известно, что будет с продуктом. Вывод на рынок. Инвестиции
- «Восходящая звезда»
 - Начались и хорошо идут продажи. Есть доход, нет прибыли
- «Дойная корова»
 - Идут продажи, но не надо прикладывать много усилий. Нового дохода не так много, но много прибыли
- «Кусачая собака»
 - Каждая продажа не дает, а отнимает деньги. Мало дохода и есть убытки



На слайде представлена Матрица БКГ (BCG Matrix: Growth–sharematrix) – инструмент стратегического портфельного анализа положения на рынке товаров, компаний и подразделений исходя из их рыночного роста и занимаемой доли на рынке. Такой инструмент, как матрица БКГ находит в настоящее время широкое применение и в менеджменте, и в маркетинге, и в других сферах экономики (и не только). Матрица БКГ была разработана экспертами Бостонской консалтинговой группы («Boston Consulting Group»), занимающейся управленческим консалтингом, в конце 1960-х годов, под руководством Брюса Хендерсона. Именно этой компании матрица обязана своим названием. Кроме того, матрица Бостонской Консалтинговой Группы стала одним из первых инструментов портфельного анализа. Для чего нужна матрица БКГ компании? Являясь простым, но эффективным

инструментом, она позволяет выявить наиболее перспективные и, напротив, самые «слабые» продукты или подразделения предприятия. Построив матрицу БКГ, менеджер или маркетолог получает наглядную картину, на основе которой он может принять решение о том, какие товары, подразделения, ассортиментные группы стоит развивать и беречь, а какие следует ликвидировать.

Обзор моделей ЖЦ:

- Модель Build-and-Fix
- Водопадная модель
- Модель быстрого прототипирования
- Инкрементная модель
- Модель синхронизации и стабилизации
- Спиральная модель
- ОО-модель

Теперь мы познакомимся с моделями ЖЦ ПО.

Какие особенности ЖЦ можно выделить уже в первом приближении для конкретных моделей? Модель Build-and-fix — это модель неполного ЖЦ, которая пригодна для малых проектов около 1000 строк и абсолютно непригодна для больших и сложных проектов с большим потенциалом развития. Водопадная, или каскадная, модель обеспечивает хорошую обратную связь с ранними стадиями ЖЦ, поскольку завершается подготовкой документов, которые позволяют перейти к следующей стадии. Без этих документов, без корректного закрытия предыдущей стадии невозможно начало следующей. Быстрое прототипирование — несамостоятельная модель, не приводящая к созданию надежного кода. Инкрементальная модель всегда дает возможность получить на каждом этапе готовый продукт, пусть и неполнофункциональный. Модель синхронизации и стабилизации, или модель Microsoft, нацелена на раннее выявление ошибок. Спиральная модель подразумевает несколько итераций и нацелена на анализ рисков. Объектно-ориентированная модель — это итеративное проектирование с перекрытием фаз и наложением их друг на друга.ф

Выводы

- Анализ ЖЦ обязателен для сложных проектов
- Знание жизненного цикла помогает в решении задач анализа, планирования и управления разработкой ПО
- Модель определяет архитектуру проекта!
- Модель определяет экономику проекта!
- Процесс разработки корпоративной системы — задача многофакторной оптимизации

- Существуют стадии ЖЦ ПО, не зависящие от методологий.

Подведем итоги модуля. Мы познакомились с подтемами: ЖЦ разработки ПО - особенности, Цели изучения ЖЦ, Цель и основные факторы разработки Экономика ЖЦ программного продукта, Стадии ЖЦ ПО.

Некоторые выводы мы уже можем сделать сейчас:

В общем смысле, ЖЦ – это изменение состояния объекта - его свойств и характеристик - во временном диапазоне: от начала его существования и до момента исчезновения. Также ЖЦ - последовательное уточнение ПО от концепции - к работающему продукту.

Анализ ЖЦ обязателен для сложных проектов. Анализ нужен всегда, где мы имеем дело со сложным объектом, который сразу осознать нельзя: мы разделяем объект на более мелкие части, которые проще понимать, и которыми легче оперировать.

Знание жизненного цикла помогает в решении задач анализа, планирования и управления разработкой ПО

Цель разработки ПО:

выбор методологии путем многомерной оптимизации с учетом, прежде всего, следующих факторов:

- сроков;
- стоимости;
- качества;
- сопровождаемости

Процесс разработки корпоративной системы – задача многофакторной оптимизации

Главное, что нам нужно запомнить:

Анализ ЖЦ обязателен для сложных проектов

В общем смысле, ЖЦ – это изменение состояния объекта (свойства и характеристики объекта) во временном диапазоне: от начала его существования и до момента исчезновения.

Есть стадии ЖЦ ПО, не зависящие от методологий.

Модель определяет архитектуру проекта!

Модель определяет экономику проекта!

Модель должна быть адекватна опыту проектной команды!

Нет универсальной модели!

Модели можно комбинировать!

Все модели имеют преимущества и недостатки!

Преимущества и недостатки имеют смысл только в контексте проекта!

Этим, однако, наше рассмотрение темы «Основы жизненного цикла» не ограничится. В следующем модуле мы расскажем об Ограниченных моделях ЖЦ.